

UNIVERSIDADE FEDERAL RURAL DO RIO DE JANEIRO
INSTITUTO MULTIDISCIPLINAR

FELIPE RANGEL DA SILVA
GABRIEL PEREZ VARGAS DE VASCONCELOS

**Chatbot para Atas Institucionais da
UFRRJ**

Prof. Ubiratam Carvalho de Paula
Jr, D.Sc.
Orientador

Prof. Fellipe Ribeiro Duarte, D.Sc.
Co-orientador

Nova Iguaçu, Junho de 2026

Chatbot para Atas Institucionais da UFRRJ

Felipe Rangel da Silva

Gabriel Perez Vargas de Vasconcelos

Projeto Final de Curso submetido ao Departamento de Ciência da Computação do Instituto Multidisciplinar da Universidade Federal Rural do Rio de Janeiro como parte dos requisitos necessários para obtenção do grau de Bacharel em Ciência da Computação.

Apresentado por:

Felipe Rangel da Silva

Gabriel Perez Vargas de Vasconcelos

Aprovado por:

Prof. Ubiratam Carvalho de Paula Jr, D.Sc.

Prof. Fellipe Ribeiro Duarte, D.Sc.

Prof. Bruno Jose Dembogurski, D.Sc.

Prof. Marcel William Rocha da Silva, D.Sc.

NOVA IGUAÇU, RJ - BRASIL

Junho de 2026

RESUMO

Chatbot para Atas Institucionais da UFRRJ

Felipe Rangel da Silva e Gabriel Perez Vargas de Vasconcelos

Junho/2026

Orientador: Ubiratam Carvalho de Paula Jr, D.Sc.

Este trabalho propõe um chatbot baseado em regras para auxiliar discentes, docentes e o público acadêmico em geral na busca de atas institucionais da Universidade Federal Rural do Rio de Janeiro (UFRRJ). O crescimento contínuo do volume de atas torna a busca manual cada vez mais custosa e para resolver esse problema, propõe-se um roteiro guiado de diálogo que conduz o usuário por um conjunto fechado de critérios de busca, sendo eles: departamento, disciplina, docentes, discentes, período, processo/edital e progressão funcional, além de um campo de assuntos gerais, traduzindo cada resposta em um parâmetro estruturado de uma consulta ao Elasticsearch. A solução foi implementada na plataforma Botpress v12, integrada a um indexador desenvolvido em Node.js responsável por extrair o conteúdo de arquivos PDF e montar dinamicamente a consulta em formato Query DSL, além de um bot para o Discord como canal alternativo de interação. Os documentos retornados são ranqueados pelo algoritmo BM25 e são apresentadas ao usuário as 3 atas com maior relevância. Os resultados indicam que o roteiro guiado de diálogo constitui uma solução viável para a geração de consultas estruturadas a partir de interações em linguagem natural, deixando claro o que pode ou não ser buscado e dispensando do usuário qualquer conhecimento sobre a organização do índice ou a sintaxe de consulta do motor de busca.

Palavras-chave: chatbot baseado em regras; recuperação da informação; Elasticsearch; atas institucionais; Botpress.

ABSTRACT

Chatbot para Atas Institucionais da UFRRJ

Felipe Rangel da Silva and Gabriel Perez Vargas de Vasconcelos

Junho/2026

Advisor: Ubiratam Carvalho de Paula Jr, D.Sc.

This work proposes a rule-based chatbot to assist students, faculty, and the broader academic community in searching institutional meeting minutes (atas) of the Federal Rural University of Rio de Janeiro (UFRRJ). The continuous growth in the volume of minutes makes manual search increasingly costly, and to address this problem, a guided dialogue script is proposed that walks the user through a closed set of search criteria, namely: department, course, faculty members, students, time period, process/notice number, and academic progression, plus a general-subject field, translating each answer into a structured parameter of a query sent to Elasticsearch. The solution was implemented on the Botpress v12 platform, integrated with an indexer built in Node.js responsible for extracting content from PDF files and dynamically assembling the query in Query DSL format, along with a Discord bot offered as an alternative interaction channel. Returned documents are ranked using the BM25 algorithm, and the 3 most relevant minutes are presented to the user. The results indicate that the guided dialogue script constitutes a viable solution for generating structured queries from natural language interactions, explicitly delimiting what can and cannot be searched while requiring no knowledge from the user about the index's organization or the search engine's query syntax.

Keywords: rule-based chatbot; information retrieval; Elasticsearch; institutional meeting minutes; Botpress.

Sumário

Resumo	i
Abstract	ii
1 Introdução	1
1.1 Objetivo	2
1.2 Organização do Trabalho	2
2 Fundamentação	4
2.1 Chatbots	4
2.1.1 Definição	4
2.1.2 Origem e Evolução dos Chatbots	5
2.1.3 Tipos de Chatbots	7
2.1.3.1 Chatbots Baseados em Menus ou Botões	7
2.1.3.2 Chatbots Baseados em Regras	7
2.1.3.3 Chatbots Impulsionados por IA	8
2.1.3.4 Voice Chatbots	8
2.1.3.5 Chatbots de IA Generativa	8

2.1.3.6	Chatbots Híbridos	9
2.1.4	Chatbots Baseados em Regras	9
2.1.5	Comparativo entre Tipos de Chatbot e os Requisitos do Projeto	10
2.2	Busca e Recuperação da Informação	12
2.2.1	Definição	12
2.2.2	Modelos de Recuperação da Informação	13
2.2.2.1	Modelo Booleano	13
2.2.2.2	Modelo Vetorial (Espaço Vetorial)	13
2.2.2.3	Modelo Probabilístico	14
2.2.2.4	Situando o BM25 e a Busca Semântica	14
2.2.3	O Problema da Busca: Indexação e Recuperação	15
3	Proposta	16
3.1	Problema: Gerando Consultas Usando Chatbots	16
3.1.1	Delimitação da Abordagem: Exclusão de Outros Tipos de Chatbot	16
3.1.2	O Papel da Consulta no Processo de Busca	20
3.1.3	O Problema de Gerar a Consulta sem Conhecer o Índice	21
3.1.4	Definindo o que Pode ou Não Ser Buscado	22
3.1.5	Solução Proposta: Roteiro Guiado de Diálogo	23
4	Implementação de um Chatbot Baseado em Regras para Recuperação de Atas da UFRRJ	25
4.1	Escolha do Chatbot	25

4.2	Arquitetura da Solução	26
4.3	Escolha do Motor de Busca	27
4.3.1	Apache Lucene	27
4.3.2	Apache Solr	27
4.3.3	Vespa	28
4.3.4	Elasticsearch	28
4.4	Indexador de Documentos	29
4.5	Mapeamento dos Critérios de Busca para a Consulta	30
4.5.1	Critérios de Busca e Variáveis de Sessão	30
4.5.2	Montagem da Consulta ao Elasticsearch	31
4.5.3	Seleção e Exibição dos Resultados	33
4.6	Integração com o Discord	34
4.7	Ambiente de Desenvolvimento	36
4.8	Utilização do Chatbot	36
4.8.1	Abertura da Conversa	36
4.8.2	Navegação pelos Critérios de Busca	37
4.8.3	Consolidação das Informações e Envio da Consulta	38
4.8.4	Exibição dos Resultados	39
5	Conclusão	41
5.1	Considerações finais	41
5.2	Limitações e trabalhos futuros	43
	Referências	45

Capítulo 1

Introdução

Na era da informação, a disseminação de novos meios de comunicação dentro do ambiente universitário, com o intuito de facilitar a comunicação entre discentes e docentes, vem se tornando cada vez mais relevante. Nesse cenário, os agentes conversacionais podem ser ferramentas importantes e ótimos intermediadores para a busca de informações de âmbito acadêmico, como respostas a FAQs relacionadas a matrículas, grade curricular e eventos, facilitando e otimizando o tempo das secretarias, que outrora possuíam o papel de entregar esse tipo de resposta.

Conforme já observado em estudos acadêmicos, tal como a monografia intitulada de *“Solução Chatbot no Ambiente Acadêmico da UFRJ”* (ARAUJO, 2020), na qual é apresentada uma solução baseada em regras dentro do ambiente acadêmico da UFRJ, focada na entrega de informações sobre matrícula, calendário acadêmico, atestados e declarações, e também no trabalho *“Um Chatbot Especializado para o Contexto da Universidade Federal de Ouro Preto”* (BARBOSA, 2023), com aplicações similares e tecnologias mais recentes, observa-se que a utilização de chatbots pode ser aplicada com esses fins.

Atualmente, na Universidade Federal Rural do Rio de Janeiro, as atas de reuniões acadêmicas armazenam informações essenciais sobre o desenvolvimento do curso e decisões acadêmicas que afetam os alunos e os professores, representando um histórico valioso para preservação e consulta.

Nesse contexto, foi observada a necessidade de uma ferramenta que facilite a busca por atas que contenham informações pertinentes sobre assuntos específicos ou buscas mais genéricas. Com fins de pesquisa e levantamento do histórico de reuniões dentro do departamento e considerando que a quantidade de atas cresce continuamente, realizar a busca manual torna-se cada vez mais difícil. Além disso, a aplicação de um sistema de consulta de atas demanda manutenção, aprendizado por parte do operador e implantação adequada ao ambiente acadêmico.

1.1 Objetivo

Diante disso, o problema central deste trabalho é permitir que perguntas realizadas por discentes, docentes ou o público acadêmico no geral sejam interpretadas e mapeadas para as atas que mais se adequam ao conteúdo de suas buscas, com o intermédio de um chatbot baseado em regras para a comunicação e um sistema de busca dentro de uma base de dados composta por essas atas.

Este trabalho tem como objetivo disponibilizar uma ferramenta conversacional, na forma de um chatbot, capaz de interpretar consultas formuladas por discentes, docentes e pelo público acadêmico em geral, com o intuito de encontrar as atas institucionais da UFRRJ mais pertinentes ao conteúdo da busca realizada.

1.2 Organização do Trabalho

O restante deste trabalho está organizado da seguinte forma: o Capítulo 2 apresenta a fundamentação teórica necessária para a compreensão da proposta, abordando a origem histórica e a evolução dos chatbots, a definição formal do termo, a taxonomia dos tipos existentes e os fundamentos de Busca e Recuperação da Informação. O Capítulo 3 descreve a proposta de solução, incluindo a arquitetura do sistema e a forma como seus componentes se relacionam para responder às consultas dos usuários. O Capítulo 4 apresenta a implementação da solução proposta, as ferramentas e tecnologias utilizadas em sua construção. Por fim, o Capítulo 5 traz as

considerações finais do trabalho, suas limitações e sugestões para trabalhos futuros.

Capítulo 2

Fundamentação

Este capítulo apresenta os conceitos teóricos necessários para a compreensão da proposta descrita no Capítulo 3. Inicialmente, é apresentada a discussão sobre chatbots, abordando sua origem histórica, a definição formal do termo, a taxonomia dos tipos existentes e a justificativa técnica para a escolha de um chatbot baseado em regras neste trabalho. Em seguida, são apresentados os fundamentos de Busca e Recuperação da Informação (BRI), incluindo os principais modelos de recuperação e o papel da indexação e da busca no funcionamento do Elasticsearch, motor utilizado no sistema proposto.

2.1 Chatbots

2.1.1 Definição

A IBM define chatbot como um programa de computador desenvolvido para simular a conversação humana com o usuário final (IBM, 2025). Essa definição é compatível com a literatura acadêmica sobre o tema. Dale (DALE, 2016) define chatbot, de forma ampla, como qualquer aplicação de software capaz de manter um diálogo com um ser humano por meio de linguagem natural, independentemente de o sistema ser chamado de assistente digital, interface conversacional ou chatbot. O conceito subjacente é sempre o de alcançar um resultado por meio de uma interação

dialógica com uma máquina.

Já Adamopoulou e Moussiades (ADAMOPOULOU; MOUSSIADES, 2020) caracterizam os chatbots como sistemas de diálogo natural que simulam conversas humanas, podendo ser implementados essencialmente por meio de duas abordagens: a correspondência de padrões (*pattern matching*), em que a resposta é selecionada a partir de regras pré-definidas que casam com a entrada do usuário, e o aprendizado de máquina, em que o sistema aprende a gerar ou selecionar respostas a partir de grandes volumes de dados de treinamento. Essa distinção entre abordagem baseada em regras e abordagem baseada em aprendizado de máquina é a base da taxonomia apresentada a seguir.

2.1.2 Origem e Evolução dos Chatbots

A história dos chatbots teve início na década de 1960, através do desenvolvimento do programa ELIZA, idealizado por Joseph Weizenbaum no Massachusetts Institute of Technology (MIT). ELIZA foi um chatbot projetado para simular uma conversa com um psicoterapeuta, funcionando com base em regras simples de construção de palavras e scripts pré-definidos (WEIZENBAUM, 1966). ELIZA não compreendia de fato o conteúdo das mensagens, mas conseguia simular interações humanas de maneira revolucionária para a época, sendo sua criação um marco do início da exploração de agentes conversacionais no campo da inteligência artificial (IA) e da linguística computacional.

Outro marco na evolução dos chatbots ocorreu em 1972, quando o psiquiatra Kenneth Colby, na Universidade de Stanford, desenvolveu o PARRY, modelo que, diferentemente do ELIZA, simulava o comportamento de um paciente com esquizofrenia paranoide. O PARRY foi submetido a uma versão do Teste de Turing (TURING, 1950), sendo avaliado por psiquiatras que interagiram com o programa e analisaram suas transcrições. Os especialistas identificaram corretamente cerca de 48% das trocas, valor que demonstra que, mesmo para profissionais da área, o PARRY se passava, na maioria das situações, por um paciente humano (COLBY et al., 1972). Essa capacidade de simulação evidencia o avanço no uso de agentes

conversacionais como ferramentas de investigação psicológica, mesmo em uma época em que a tecnologia não era tão avançada quanto hoje, reforçando o PARRY como uma etapa importante na história da IA conversacional.

Após os primeiros experimentos com o ELIZA e o PARRY, o desenvolvimento de chatbots avançou consideravelmente com a criação do *Artificial Linguistic Internet Computer Entity* (A.L.I.C.E), em 1995, por Richard Wallace. Diferente de seus predecessores, A.L.I.C.E foi projetado com o uso de uma linguagem específica chamada AIML (*Artificial Intelligence Markup Language*), que permitiu a construção de padrões de entrada e respostas de forma modular e escalável, facilitando sua expansão e adaptação para diferentes tipos de diálogos (WALLACE, 2009). Mesmo sem utilizar técnicas de aprendizado de máquina, sua estrutura baseada em regras, somada ao grande volume de dados escritos manualmente, possibilitava interações elaboradas, tendo A.L.I.C.E conquistado reconhecimento internacional após vencer três vezes o Prêmio Loebner (2000, 2001 e 2004), competição baseada no Teste de Turing (TURING, 1950) que avalia o grau de humanização de um chatbot (SHUM; HE; LI, 2018).

A partir de 2011, com o lançamento da Siri pela Apple, integrada ao iPhone 4S, iniciou-se uma nova era no desenvolvimento de chatbots inteligentes, marcando a popularização dos assistentes pessoais inteligentes no cotidiano dos usuários. A Siri foi pioneira ao trazer uma interface de voz capaz de compreender comandos naturais e executar tarefas diversas, integrando múltiplas fontes de dados e sensores embarcados no dispositivo (HOY, 2018). Ao longo da década seguinte surgiram diversos outros assistentes pessoais inteligentes, como a Cortana da Microsoft e a Alexa da Amazon, ambas em 2014, e o Google Assistant, em 2016. Esses agentes são capazes de integrar informações provenientes de múltiplos sensores, como localização, clima, movimento, toques e gestos, além de acessarem diversas fontes de dados pessoais, como músicas, calendários, e-mails e perfis sociais (SHUM; HE; LI, 2018).

Mais recentemente, a evolução dos chatbots alcançou um novo patamar com a adoção de modelos de linguagem de grande escala, os LLMs (*Large Language Models*) (IBM, 2023a). Ao contrário dos chatbots mais tradicionais, que funcionam com base

em fluxos fixos ou regras pré-programadas, os LLMs são treinados com enormes volumes de texto por meio de técnicas de *Deep Learning* e conseguem lidar com tarefas bem mais complexas, como responder perguntas abertas, escrever textos longos e bem estruturados, resumir documentos, traduzir idiomas e explicar conceitos técnicos. Essa flexibilidade e capacidade de adaptação a diferentes situações tornam esses modelos especialmente úteis em áreas como educação, atendimento ao cliente, saúde e produtividade no ambiente corporativo (BOMMASANI; AL., 2021). A Seção 2.1.3 situa essa abordagem dentro da taxonomia de chatbots, e a Seção 2.2 retoma os LLMs sob a perspectiva da Recuperação da Informação, relacionando-os ao modelo probabilístico apresentado a seguir.

2.1.3 Tipos de Chatbots

A IBM, por meio da publicação “*Seis tipos de chatbots e como escolher o ideal para sua empresa*” (IBM, 2025), descreve seis categorias de chatbots disponíveis atualmente, apresentadas a seguir.

2.1.3.1 Chatbots Baseados em Menus ou Botões

Representam o nível mais básico entre os chatbots. A interação com o usuário se dá pela seleção de opções em um script que se encaixa melhor nas suas necessidades, podendo haver mais opções conforme as escolhas forem sendo feitas, até chegar à opção mais adequada. Esses chatbots operam como árvores de decisão e são bons para tarefas transacionais, sendo focados na resolução de funcionalidades simples e úteis para responder perguntas repetitivas, previsíveis e diretas. A dificuldade aparece em solicitações menos genéricas, pois as opções de resposta são pré-definidas e limitadas. Caso a necessidade do usuário não esteja disponível no menu, o chatbot se torna inútil.

2.1.3.2 Chatbots Baseados em Regras

Possuem a funcionalidade da árvore de decisão, assim como os chatbots baseados em menu, adicionando também regras de lógica condicional para o desenvolvimento

de fluxos de automação de conversa. Atuam essencialmente com FAQs interativas, necessitando de uma quantidade de combinações pré-definidas de perguntas e respostas para a interação com o usuário. Operam com detecção básica de palavras-chave, sendo fáceis de treinar e funcionando bem quando recebem perguntas dentro de um escopo pré-definido, mas têm dificuldade para lidar com perguntas mais complexas e não previstas no escopo.

2.1.3.3 *Chatbots Impulsionados por IA*

Possuem capacidade de compreensão das perguntas do usuário com recursos de IA e Entendimento de Linguagem Natural (NLU), conseguindo detectar rapidamente as informações relevantes a partir das interações com o usuário, tornando a interação mais conversacional. Possuem a possibilidade de buscar mais informações com o usuário para o entendimento correto do escopo fornecido e solicitado, podendo apresentar opções de resposta entre as quais o usuário escolhe a que mais se encaixa no contexto encontrado. Utilizam algoritmos de aprendizado de máquina em sua implementação, permitindo entendimento, aprendizagem e desenvolvimento contínuo.

2.1.3.4 *Voice Chatbots*

São ferramentas de conversação que permitem a interação falada como alternativa à interação textual, podendo oferecer as mesmas funcionalidades dos chatbots de IA convencionais, mas implementadas com tecnologia *Text-to-Speech* e *Speech-to-Text*. Com a aplicação do Processamento de Linguagem Natural e interações com tecnologias de computação e telefonia, os chatbots por voz conseguem compreender perguntas faladas, analisar a intenção do usuário e apresentar respostas em tom de conversa, aumentando o engajamento e a imersão.

2.1.3.5 *Chatbots de IA Generativa*

Oferecem funcionalidades mais avançadas que os anteriores, com maior fluência na compreensão da linguagem comum e maior adaptação à linguagem do usuário, podendo gerar conteúdo novo como resultado sendo texto, imagens ou áudio, com

base em *Large Language Models* (LLMs), nos quais são treinados. Apresentam maior personalização aos usuários, aumentando o alcance e a velocidade no atendimento. As interfaces de chatbots com IA generativa podem reconhecer, resumir, traduzir, prever e criar conteúdo em resposta à consulta de um usuário.

2.1.3.6 Chatbots Híbridos

São sistemas de IA conversacional que combinam lógica baseada em regras e recursos baseados em aprendizado de máquina, proporcionando uma experiência mais versátil ao lidar com tarefas de diferentes níveis de complexidade. Funcionam com um conjunto de regras e scripts pré-definidos, que dão estrutura ao diálogo, enquanto a camada de IA contribui com potencial de aprendizado para interações mais complexas.

2.1.4 Chatbots Baseados em Regras

Chatbots baseados em regras operam por meio de um conjunto fechado de padrões de entrada e saída definidos previamente pelo desenvolvedor, sem capacidade de generalização para situações fora desse conjunto. Adamopoulou e Moussiades (ADAMOPOULOU; MOUSSIADES, 2020) descrevem essa abordagem como *pattern matching*: a entrada do usuário é comparada a padrões pré-estabelecidos e, ao haver correspondência, uma resposta associada a esse padrão é selecionada. Essa lógica está na base de linguagens historicamente usadas para construção de chatbots, como a AIML (*Artificial Intelligence Markup Language*), empregada no A.L.I.C.E (WALLACE, 2009), apresentado na Seção 2.1.2.

Singh, Joesph e Jabbar (SINGH; JOESPH; JABBAR, 2019) descrevem um chatbot baseado em regras desenvolvido para responder a consultas administrativas de estudantes de uma universidade (o “APU Admin Bot”), no qual a correspondência de padrões como palavras, frases e até ações específicas do usuário aciona um conjunto pré-definido de respostas, dispensando a necessidade de programação tradicional ou de grandes volumes de dados de treinamento. Esse caso ilustra a adequação da abordagem baseada em regras a domínios fechados, com escopo de perguntas relativa-

mente previsível, característica também presente no contexto das atas institucionais tratadas neste trabalho.

2.1.5 Comparativo entre Tipos de Chatbot e os Requisitos do Projeto

A escolha do tipo de chatbot a ser utilizado neste trabalho deve considerar as características do problema apresentado: um sistema de busca em um conjunto fechado de documentos, sendo eles as atas da UFRRJ, sem necessidade de geração de conteúdo novo, com baixo orçamento disponível para licenciamento de ferramentas e que precisa se integrar a um motor de busca textual (Elasticsearch (IBM, 2023b)) por meio de chamadas a uma API. A Tabela 2.1 resume, para cada tipo de chatbot apresentado na Seção 2.1.3, a adequação a esses requisitos.

Tabela 2.1: Comparativo entre tipos de chatbot e os requisitos do projeto

Tipo	Adequação ao escopo fechado	Custo de licenciamento	Integração com API externa (Elasticsearch)
Baseado em menus/botões	Limitado: difícil mapear os critérios de busca em menus rígidos	Baixo/nenhum	Possível, mas pouco flexível para parâmetros livres
Baseado em regras	Alto: escopo fechado favorece fluxos com regras condicionais bem definidas	Baixo/nenhum (Botpress v12 open source)	Suporta chamadas HTTP/JS dentro do fluxo
Impulsionado por IA / NLU	Alto poder de generalização, mas exige treinamento de modelos de NLU e maior esforço de configuração	Médio/alto (modelos proprietários)	Possível, porém com custo adicional
Voice chatbot	Não se aplica ao caso de uso (interação textual)	Médio/alto	Possível, mas fora do escopo
IA generativa	Alta flexibilidade, porém risco de respostas não fundamentadas nas atas (alucinação) e custo recorrente de uso de LLM	Alto (APIs de LLM pagas)	Possível, mas exige camada adicional de RAG
Híbrido	Combina regras e IA, mas adiciona complexidade de manutenção desnecessária para o escopo atual	Médio	Possível

Diante desses requisitos, optou-se pelo chatbot baseado em regras, implementado

na plataforma Botpress v12 (Botpress, 2025a; Botpress, 2025b). Essa escolha decorre de três fatores combinados: (i) o escopo do problema é fechado e o usuário busca atas dentro de um conjunto de critérios pré-definidos (departamento, disciplina, docentes, discente, data, processo/edital/ata, progressão funcional e outros assuntos), o que se encaixa em um fluxo de diálogo guiado por regras; (ii) o Botpress v12 é uma ferramenta de código aberto, sem custo de licenciamento, podendo ser hospedada localmente (Botpress, 2025a); e (iii) a plataforma permite a execução de ações em JavaScript dentro do fluxo de conversa, viabilizando a integração com a API REST do indexador Node.js, que por sua vez consulta o Elasticsearch (IBM, 2023b). Chatbots impulsionados por IA, de IA generativa ou híbridos poderiam oferecer maior flexibilidade na interpretação de perguntas livres, mas introduziriam custo de licenciamento de modelos de linguagem e risco de geração de respostas não embasadas no conteúdo das atas.

2.2 Busca e Recuperação da Informação

2.2.1 Definição

Recuperação de Informação (RI), do inglês *Information Retrieval*, é definida por Manning, Raghavan e Schütze (MANNING; RAGHAVAN; SCHÜTZE, 2008) como a atividade de localizar, dentro de uma grande coleção, material de natureza não estruturada que satisfaça uma necessidade de informação expressa por uma consulta. Baeza-Yates e Ribeiro-Neto (BAEZA-YATES; RIBEIRO-NETO, 2010) complementam essa definição ao descrever a RI como a área responsável por representar, armazenar, organizar e acessar itens de informação de forma a facilitar o acesso do usuário às informações de seu interesse, lidando com a representação, o armazenamento e a recuperação de informação textual.

Diferentemente de uma consulta a um banco de dados estruturado, em que a correspondência entre consulta e registros é exata, a RI trata do problema da relevância: dado um conjunto de documentos e uma consulta, o sistema deve estimar quais documentos são relevantes para a necessidade de informação do usuário,

ordenando os resultados por algum critério de pontuação (MANNING; RAGHAVAN; SCHÜTZE, 2008).

2.2.2 Modelos de Recuperação da Informação

Baeza-Yates e Ribeiro-Neto (BAEZA-YATES; RIBEIRO-NETO, 2010) organizam os modelos clássicos de RI em três grandes famílias: o modelo booleano, o modelo vetorial (espaço vetorial) e o modelo probabilístico.

2.2.2.1 Modelo Booleano

Trata-se de um modelo de recuperação de informação baseado na teoria dos conjuntos e na álgebra booleana, neste modelo documentos e consultas são representados como conjuntos de termos combinados por operadores lógicos (*AND*, *OR*, *NOT*). Um documento é considerado relevante se satisfaz exatamente a expressão booleana da consulta, sem que exista uma noção de grau de relevância, ou seja, o documento ou corresponde ou não corresponde a busca realizada. (BAEZA-YATES; RIBEIRO-NETO, 2010).

2.2.2.2 Modelo Vetorial (Espaço Vetorial)

No modelo de espaço vetorial, proposto originalmente por Salton, Wong e Yang (SALTON; WONG; YANG, 1975), documentos e consultas são representados como vetores em um espaço de n dimensões, em que cada dimensão corresponde a um termo do vocabulário. O grau de similaridade entre uma consulta e um documento é calculado por uma medida de distância ou similaridade entre os vetores, permitindo ordenar os documentos por um grau contínuo de relevância, em vez da resposta binária do modelo booleano. Esse modelo é a base conceitual tanto de esquemas de ponderação clássicos, como o TF-IDF, quanto das abordagens modernas de busca semântica baseadas em *embeddings*, nas quais documentos e consultas são representados como vetores densos aprendidos por modelos de linguagem, e a similaridade vetorial captura relações semânticas entre termos que vão além da correspondência exata de palavras.

2.2.2.3 Modelo Probabilístico

O modelo probabilístico estima a probabilidade de um documento ser relevante para uma consulta, ordenando os resultados de acordo com essa estimativa (BAEZAYATES; RIBEIRO-NETO, 2010). A família de funções de ranqueamento BM25 (*Best Matching 25*) é a realização mais difundida desse modelo, derivada do arcabouço probabilístico de relevância desenvolvido por Robertson e colaboradores (ROBERTSON; ZARAGOZA, 2009). O BM25 pondera a frequência de cada termo da consulta no documento, a frequência inversa do termo na coleção (termos raros contribuem mais para a pontuação que termos comuns) e o comprimento do documento, produzindo uma pontuação de relevância para cada documento em relação à consulta. É esse o algoritmo utilizado pelo Elasticsearch para o ranqueamento padrão de resultados (IBM, 2023b).

A noção de estimativa de probabilidade que sustenta o modelo probabilístico de RI também é o princípio por trás dos LLMs (*Large Language Models*) que impulsionam os chatbots de IA generativa apresentados na Seção 2.1.3. Segundo a IBM (IBM, 2023a), um LLM é um modelo de aprendizado de máquina aplicado à IA generativa, treinado com grandes volumes de texto para compreender e gerar linguagem de forma contextualmente coerente. Do ponto de vista matemático, um LLM estima, a cada passo, a probabilidade de cada token do vocabulário ser o próximo em uma sequência, dado o contexto precedente; a representação vetorial desse contexto (*embeddings*) segue a mesma lógica de espaço vetorial discutida na Seção 2.2.2.2. Há, portanto, uma proximidade matemática entre os dois modelos, pois ambos se apoiam em estimativas de probabilidade, ainda que aplicados a fins distintos: o modelo probabilístico de RI estima a relevância de um documento para uma consulta, com o objetivo de ranqueamento, enquanto o LLM estima a probabilidade do próximo token em uma sequência, com o objetivo de geração de linguagem.

2.2.2.4 Situando o BM25 e a Busca Semântica

No sistema proposto neste trabalho, a recuperação textual sobre o conteúdo das atas é realizada pelo Elasticsearch, que utiliza o BM25 como função de ranqueamento

padrão (IBM, 2023b). A busca por similaridade semântica, já mencionada neste capítulo como mecanismo complementar ao BM25, situa-se dentro da família de modelos vetoriais: tanto a consulta quanto os documentos podem ser representados como vetores de *embeddings*, e a relevância é estimada pela proximidade entre esses vetores no espaço semântico, capturando relações de significado que o BM25, por operar sobre a correspondência lexical de termos, não captura diretamente.

2.2.3 O Problema da Busca: Indexação e Recuperação

O funcionamento de um sistema de RI pode ser dividido em duas etapas principais: a indexação e a busca propriamente dita (MANNING; RAGHAVAN; SCHÜTZE, 2008).

A indexação consiste em processar previamente a coleção de documentos para construir uma estrutura de dados que viabilize buscas eficientes sem a necessidade de varrer o conteúdo integral de cada documento a cada consulta. No caso do Elasticsearch, essa estrutura é o índice invertido, no qual cada termo do vocabulário é associado à lista de documentos em que ocorre, junto a estatísticas usadas pelo BM25 (frequência do termo, frequência do documento, comprimento do documento) (IBM, 2023b). No sistema proposto, essa etapa corresponde ao processo de extração de texto dos PDFs das atas e ao envio desse conteúdo ao Elasticsearch para indexação, associando a cada documento campos estruturados como título e data da ata, além do conteúdo textual completo.

A busca, por sua vez, consiste em receber uma consulta, processá-la e utilizá-la para percorrer o índice em busca dos documentos mais relevantes, retornando-os ordenados por pontuação. No sistema proposto, a consulta enviada ao Elasticsearch é montada a partir dos critérios informados pelo usuário através do chatbot, e os documentos retornados são ordenados de forma decrescente pelo *score* atribuído pelo BM25, sendo selecionados os três melhores resultados para apresentação ao usuário. O Capítulo 3 discute, em nível conceitual, como o chatbot resolve a desconexão entre o vocabulário livre do usuário e a organização do índice na geração dessa consulta; o Capítulo 4 apresenta os detalhes de implementação dessa etapa.

Capítulo 3

Proposta

Este capítulo apresenta o problema central enfrentado na construção de um chatbot de busca sobre as atas institucionais da UFRRJ, com a geração de consultas adequadas ao motor de busca a partir da interação em linguagem natural com o usuário e a solução proposta para esse problema, baseada em um roteiro guiado de diálogo que traduz as respostas do usuário em parâmetros estruturados de uma consulta ao Elasticsearch.

3.1 Problema: Gerando Consultas Usando Chatbots

3.1.1 Delimitação da Abordagem: Exclusão de Outros Tipos de Chatbot

A Tabela 2.1 resume a adequação geral de cada tipo aos requisitos técnicos do projeto; a Tabela 3.1 detalha, de forma mais direta, o motivo específico pelo qual cada um dos cinco tipos não escolhidos foi descartado, à luz do contexto de busca textual sobre as atas institucionais da UFRRJ e da restrição a ferramentas com plano gratuito.

Tabela 3.1: Motivos para a não adoção dos demais tipos de chatbot

Tipo	Motivo de não adoção
Baseado em menus/botões	Os critérios de busca admitem texto livre (como nomes ou períodos por extenso). Um chatbot de menus, restrito a opções fixas, exigiria um número impraticável de ramificações, tornando-se inviável para este escopo.
Impulsionado por IA / NLU	Exige o treinamento de um modelo específico de NLU para reconhecer os critérios de busca, demandando maior esforço de configuração. Além disso, plataformas gratuitas com esse recurso costumam impor limites de uso mais restritivos do que o Botpress v12.
Voice chatbot	O canal de interação previsto é exclusivamente textual (Botpress e Discord). Não há demanda por interação falada no contexto das atas da UFRRJ que justifique a complexidade de adotar tecnologias de <i>Speech-to-Text</i> e <i>Text-to-Speech</i> .
IA generativa	Modelos de linguagem (LLMs) introduzem o risco de alucinação (respostas não embasadas), o que é inadequado para um sistema institucional que exige precisão. Além disso, o custo de APIs por <i>tokens</i> conflita com a restrição de uso de plataformas gratuitas.
Híbrido	Por mesclar regras e aprendizado de máquina, herda os riscos de respostas desalinhadas e o maior esforço de configuração e manutenção da IA, sem trazer benefícios claros para um escopo de busca já bem definido, onde o fluxo de regras é suficiente.

Cabe aqui o debate, olhando especificamente para chatbots com algum nível de utilização de IA, olhar para o ângulo específico do problema de geração de consultas: o risco prático de uma resposta não embasada no conteúdo real das atas.

No sistema proposto, a resposta apresentada ao usuário deve corresponder a um documento institucional existente que é a ata recuperada do índice. Um chatbot de IA generativa, ao gerar texto livremente a partir de um modelo de linguagem, pode produzir uma resposta gramaticalmente correta e aparentemente segura, mas sem garantia de que seu conteúdo corresponda ao que está de fato registrado nas atas. Esse tipo de erro, conhecido como alucinação, é extremamente grave neste domínio: o usuário poderia receber uma informação inexistente ou incorreta sobre uma decisão acadêmica real, como a data de uma reunião, o resultado de uma progressão funcional ou o teor de um processo.

Esse risco já se concretizou fora do contexto acadêmico, em uma disputa judicial real. Em fevereiro de 2024, o *British Columbia Civil Resolution Tribunal*, no Canadá, responsabilizou a empresa aérea Air Canada porque seu chatbot informou incorretamente a um passageiro que ele poderia solicitar, de forma retroativa, uma tarifa reduzida por luto familiar — política que a empresa, na realidade, não permitia. O tribunal rejeitou o argumento de que o chatbot seria uma entidade distinta da empresa e determinou que a Air Canada honrasse a informação incorreta fornecida por seu próprio sistema (CBC News, 2024; American Bar Association, 2024). O caso ilustra, de forma concreta, que uma resposta tecnicamente convincente, mas não embasada em fonte confiável, tem consequências reais para quem a recebe e para quem a fornece.

Esse argumento reforça, no contexto deste trabalho, a opção por um chatbot baseado em regras combinado a uma busca textual determinística sobre o Elasticsearch: o sistema proposto só pode retornar como resposta o que de fato está indexado a partir do conteúdo das atas, sem gerar texto novo.

Desta forma, trazemos uma análise dentro do universo de chatbots baseados em regras, onde ainda existem diversas plataformas gratuitas disponíveis, cada uma com características distintas quanto ao modelo de hospedagem, ao limite de uso

oferecido no plano gratuito e à curva de aprendizado exigida para sua configuração. A Tabela 3.2 resume as principais alternativas consideradas e os motivos pelos quais não foram adotadas neste trabalho.

Tabela 3.2: Comparativo entre chatbots de regra gratuitos

Ferramenta	Limite do plano gratuito	Observação
Botpress v12	Ilimitado — versão <i>open source</i> , hospedada localmente	Ferramenta escolhida: sem limite de uso, permite execução de ações em JavaScript dentro do fluxo
Rasa	Ilimitado — versão <i>open source</i> , hospedada localmente	Mais técnico: configuração via código e arquivos YAML, com curva de aprendizado maior que a do Botpress
Rocket.Chat	Ilimitado — versão <i>open source</i> , hospedada localmente	Plataforma de chat corporativo, e não um framework de chatbot dedicado; a construção de fluxos de regras depende de <i>apps</i> e <i>scripts</i> adicionais, exigindo configuração mais técnica
Dialogflow ES	Limitado por cota de requisições por minuto (180 req./min na edição de avaliação)	Boa integração com diversos canais (sites, WhatsApp, Messenger), porém de propriedade do Google, sem opção de hospedagem própria
Flow XO	100 interações por mês	Construção de fluxos visual e de fácil utilização, porém com limite mensal de uso baixo para o volume de consultas esperado no projeto

Entre as alternativas *open source* que poderiam ser hospedadas sem custo e sem limite de uso — Botpress v12, Rasa (Rasa, 2026) e Rocket.Chat (Rocket.Chat, 2026)

—, o Botpress v12 foi escolhido por oferecer uma interface visual de construção do fluxo de diálogo aliada à possibilidade de execução de código JavaScript diretamente nos nós do fluxo, equilibrando a facilidade de uso de uma ferramenta visual com a flexibilidade de uma ferramenta orientada a código, sem o esforço de configuração mais técnico exigido pelo Rasa e pelo Rocket.Chat. Já o Dialogflow (Google Cloud, 2026) e o Flow XO (Flow XO, 2026), embora ofereçam construção de fluxos mais simples, dependem da hospedagem do próprio fornecedor e impõem limites de uso (por cota de requisições, no caso do Dialogflow, ou por número de interações mensais, no caso do Flow XO) que poderiam se tornar uma restrição ao longo do desenvolvimento e da avaliação do sistema.

3.1.2 O Papel da Consulta no Processo de Busca

Como discutido na Seção 2.2.3 do Capítulo 2, um sistema de Recuperação da Informação depende de duas etapas: a indexação prévia da coleção de documentos e a busca, que utiliza uma consulta para localizar e ranquear os documentos relevantes dentro do índice (MANNING; RAGHAVAN; SCHÜTZE, 2008). A consulta é, portanto, a interface entre a necessidade de informação do usuário e o índice construído sobre a coleção: é a partir dela que o motor de busca seleciona quais campos consultar, com quais operadores e com qual peso cada termo deve contribuir para a pontuação final.

No sistema proposto, essa consulta não é digitada diretamente pelo usuário em uma caixa de busca livre. Em vez disso, ela é construída a partir das respostas fornecidas ao chatbot ao longo de um diálogo guiado e, somente então, repassada ao componente responsável por montar a consulta estruturada para o Elasticsearch. Essa separação entre a interação conversacional e a montagem da consulta é o ponto central da proposta: cabe ao chatbot decidir quais informações coletar e em que formato, e cabe ao motor de busca interpretar essas informações como critérios de relevância. Como cada critério respondido pelo usuário se traduz em um parâmetro adicional da consulta, quanto mais critérios informados, mais restritiva a busca, aumentando as chances de que a resposta retornada seja, de fato, a que o usuário procura.

3.1.3 O Problema de Gerar a Consulta sem Conhecer o Índice

Um chatbot baseado em regras conduz a conversa por um fluxo pré-definido, mas o conteúdo das mensagens trocadas com o usuário é, em geral, texto livre: o usuário pode digitar o nome de um professor com ou sem acentos, abreviar o nome de um departamento, descrever um período de forma textual (“de março a abril de 2026”) ou usar termos coloquiais para se referir a um assunto. Esse vocabulário não tem relação direta com a forma como o conteúdo das atas está organizado no índice do Elasticsearch, que distingue, por exemplo, o texto integral de cada documento de metadados estruturados como a data a que a ata se refere (IBM, 2023b).

Alguns exemplos concretos ilustram esse descompasso. Ao ser perguntado sobre o período da reunião, o usuário pode responder “de março a abril de 2026” em vez de informar datas em um formato estruturado; essa resposta precisa ser interpretada como um intervalo e direcionada a um filtro de data, e não ao conteúdo textual da busca. Ao ser perguntado sobre o docente envolvido, o usuário pode digitar apenas “professor que apresentou sobre TCC”, sem citar um nome próprio, misturando no mesmo campo um termo descritivo de assunto com o critério de docente. Já uma resposta como “reunião sobre progressão docente” mistura, em uma única frase, um termo relacionado ao critério de progressão funcional com uma referência genérica ao tipo de evento, sem indicar se o termo relevante para a busca é “progressão” ou a frase completa. Em todos esses casos, o texto livre fornecido pelo usuário precisa ser mapeado para o campo correto do índice — conteúdo textual ou filtro estruturado de data — e associado ao peso de relevância adequado ao critério correspondente, e não tratado de forma uniforme.

Essa desconexão entre o vocabulário livre do usuário e a organização do índice é o problema central enfrentado na geração da consulta: se cada resposta do usuário fosse simplesmente concatenada e enviada como uma única *string* de busca, o motor de busca trataria todos os termos com o mesmo peso e buscaria apenas no conteúdo textual das atas, perdendo a oportunidade de usar informações estruturadas para filtrar resultados, e de atribuir pesos diferentes a termos que, semanticamente, têm relevância distinta (por exemplo, o nome de um discente normalmente é mais

discriminativo do que uma palavra de um assunto genérico).

3.1.4 Definindo o que Pode ou Não Ser Buscado

A solução para essa desconexão começa pela delimitação do que pode ser buscado. O fluxo principal do chatbot conduz o usuário por um conjunto fechado de critérios de busca considerando aspectos recorrentes do conteúdo das atas: o departamento responsável pela reunião, a disciplina mencionada, os docentes e discentes envolvidos no assunto tratado, o período em que a ata foi produzida, o número de um processo ou edital referenciado e termos relacionados a progressões funcionais ou acadêmicas. Além desses critérios, o roteiro reserva um campo de outros assuntos, utilizado tanto para descrições livres do tema da ata quanto como destino de qualquer informação que o usuário forneça e que não se enquadre nos demais critérios. A ideia de possuir um campo aberto se faz necessária unicamente para o usuário ter a capacidade de encontrar temas relevantes no contexto acadêmico que não poderiam ser encontrados com as outras opções mapeadas (por exemplo: o usuário deseja saber sobre comissão de estágio).

Cada um desses critérios corresponde, na consulta enviada ao Elasticsearch, a um componente de busca sobre o conteúdo textual das atas, com um tipo de correspondência e um peso de relevância próprios: critérios mais específicos e discriminativos recebem maior peso e exigem correspondência mais estrita, enquanto critérios mais genéricos recebem peso menor e admitem maior tolerância a variações textuais. O período informado pelo usuário, por sua vez, não contribui para a pontuação de relevância: ele é tratado como um filtro sobre a data da ata, restringindo o conjunto de documentos candidatos sem afetar o *score* atribuído pelo BM25. Dessa forma, delimita-se de forma explícita o que pode ser buscado e como cada resposta do usuário se traduz em um componente da consulta ao Elasticsearch, fechando a lacuna entre o vocabulário livre da conversa e a organização do índice. O mapeamento detalhado de cada critério para os parâmetros de implementação correspondentes é apresentado no Capítulo 4 (Seção 4.5).

Com o escopo do roteiro de perguntas bem definido, combinado com a utilização

de um chatbot baseado em regras sem utilização de IA, o usuário fica limitado a encontrar tão somente o que for discutido nas reuniões do colegiado, sendo as atas informações públicas, disponibilizadas pela própria universidade. Esse conjunto bem definido de roteiro e tipo de chatbot delimita também o que não pode ser encontrado ou respondido pelo sistema: qualquer pergunta cujo conteúdo não esteja registrado em alguma ata é simplesmente respondida com a ausência de resultados, e não com uma tentativa de resposta gerada a partir de conhecimento externo ao índice. Um chatbot de IA generativa, por outro lado, não está sujeito a essa mesma limitação, podendo ser manipulado por um usuário para sair completamente do escopo para o qual foi projetado. Um caso real ilustra esse risco: em dezembro de 2023, o chatbot baseado em ChatGPT de uma concessionária Chevrolet em Watsonville, na Califórnia, foi induzido por um usuário, por meio de instruções inseridas na própria conversa (uma técnica conhecida como *prompt injection*), a concordar em vender um veículo Chevy Tahoe 2024 — avaliado em mais de USD 58 mil — por apenas um dólar, chegando a declarar a oferta como “juridicamente vinculante” (AI Incident Database, 2023; VentureBeat, 2023). Embora a concessionária não tenha honrado a venda, o episódio expõe como um chatbot de IA generativa, sem as restrições rígidas de um fluxo de regras, pode ser levado a agir completamente fora do escopo para o qual foi originalmente projetado. A delimitação do escopo de busca atua como uma barreira de proteção adicional: o sistema proposto não apenas direciona o usuário para o que pode ser encontrado, mas também garante que nada seja respondido além do que está de fato registrado nas atas.

3.1.5 Solução Proposta: Roteiro Guiado de Diálogo

A solução proposta consiste em um roteiro guiado de diálogo que conduz o usuário por perguntas relativas a cada um dos critérios de busca apresentados na seção anterior. Cada resposta do usuário é armazenada ao longo da conversa, sem que ele precise conhecer a forma como o conteúdo das atas está organizado no índice ou a sintaxe de consulta do Elasticsearch.

Ao final do roteiro, as respostas coletadas passam por uma etapa de consolidação,

na qual a resposta sobre o departamento é separada em nome e sigla, permitindo que a busca encontre a ata tanto pelo nome por extenso quanto pela forma abreviada, e a resposta sobre o período é interpretada como um intervalo de datas; quando o texto fornecido não pode ser interpretado como uma data, ele é redirecionado para o critério de assunto geral, evitando que a informação seja descartada. As respostas consolidadas são então traduzidas em uma consulta estruturada, na qual cada critério informado se torna um componente de busca com seu próprio peso de relevância, e o período, quando informado, se torna um filtro sobre a data da ata. Essa consulta é enviada ao motor de busca, que retorna os documentos ranqueados por BM25 (IBM, 2023b). Por fim, sobre os documentos retornados é aplicado um cálculo adicional de proporcionalidade com a fração dos termos informados pelo usuário que efetivamente aparecem no título e no trecho de cada ata, usado para reordenar os melhores resultados antes de apresentá-los.

Esse roteiro guiado torna o processo de geração da consulta transparente em dois sentidos: para o usuário, que é conduzido por perguntas em linguagem natural sem precisar conhecer a organização do índice; e para o desenvolvedor, que pode rastrear de forma direta como cada resposta do usuário se transforma em um componente da consulta enviada ao Elasticsearch, facilitando a manutenção e a extensão do sistema e tornando a adição de um novo critério de busca mais simples, sendo necessário apenas a inclusão de uma nova etapa no roteiro de diálogo e do componente de busca correspondente na consulta. A implementação concreta desse roteiro, incluindo as ferramentas utilizadas e os detalhes da consulta ao Elasticsearch, é descrita no Capítulo 4.

Capítulo 4

Implementação de um Chatbot Baseado em Regras para Recuperação de Atas da UFRRJ

Este capítulo descreve a implementação da solução proposta, apresentando as ferramentas e bibliotecas escolhidas para cada componente, a forma como eles foram desenvolvidos e o ambiente necessário para sua execução.

4.1 Escolha do Chatbot

A implementação da camada conversacional partiu de uma decisão já justificada na Seção 2.1.5: a adoção de um chatbot baseado em regras, em vez de soluções fundamentadas em NLU, IA generativa ou modelos híbridos (Tabelas 2.1 e 3.1). Essa escolha apoiou-se em três fatores combinados: o escopo fechado do problema, em que o usuário busca atas dentro de um conjunto pré-definido de critérios, favorece um fluxo de diálogo estruturado por regras condicionais; a ausência de orçamento para licenciamento de ferramentas torna preferível uma solução sem custo recorrente; e a necessidade de integração com uma API externa de busca exige suporte à execução de código dentro do próprio fluxo de conversa.

Dentro do universo de plataformas de chatbot baseado em regras, gratuitas e sem limite de uso quando hospedadas localmente, a Tabela 3.2 já comparou o Botpress v12 às alternativas Rasa, Rocket.Chat, Dialogflow e Flow XO. Diante desses critérios, optou-se pela plataforma **Botpress v12** (Botpress, 2025a; Botpress, 2025b), ferramenta de código aberto, sem custo de licenciamento, que pode ser hospedada localmente. Além de fornecer uma interface visual para construção do fluxo de diálogo, o Botpress v12 permite a execução de ações em JavaScript diretamente no fluxo, recurso explorado neste trabalho para realizar chamadas HTTP à API REST do indexador de documentos, apresentado na Seção 4.4.

As seções seguintes detalham essa implementação: a Seção 4.2 apresenta a arquitetura geral da solução; a Seção 4.3 justifica a escolha do Elasticsearch como motor de busca; a Seção 4.4 descreve o indexador de documentos desenvolvido em Node.js; a Seção 4.5 detalha o mapeamento dos critérios de busca coletados pelo chatbot para a consulta ao Elasticsearch; e as Seções 4.6 e 4.7 descrevem, respectivamente, a integração com o Discord e o ambiente de desenvolvimento necessário para a execução do sistema.

4.2 Arquitetura da Solução

A solução é composta por quatro componentes principais que se comunicam entre si: o **Botpress**, responsável pela interface conversacional e pelo fluxo de diálogo com o usuário; o **Elasticsearch**, responsável pelo armazenamento e pela busca por similaridade nas atas; o **indexador**, um servidor desenvolvido em Node.js responsável por extrair o conteúdo dos arquivos PDF e disponibilizá-lo ao Elasticsearch; e um **bot do Discord**, que oferece um canal alternativo de comunicação com o usuário, repassando as mensagens recebidas para o Botpress.

O fluxo de uma consulta ocorre da seguinte forma: o usuário envia uma pergunta pelo Botpress ou pelo Discord, que a encaminha ao indexador por meio de uma requisição HTTP; o indexador, por sua vez, monta uma consulta ao Elasticsearch e retorna os trechos das atas mais relevantes para a pergunta realizada.

4.3 Escolha do Motor de Busca

A escolha do motor de busca é uma das decisões mais relevantes neste trabalho, pois determina diretamente a forma como os documentos são indexados, consultados e ranqueados. Existem diversas soluções disponíveis para essa finalidade, cada uma com características, modelos de licenciamento e casos de uso distintos.

Abaixo falaremos brevemente dos motores de busca mais relevantes que encontramos:

4.3.1 Apache Lucene

O **Apache Lucene** (Apache Software Foundation, 2025a) é uma biblioteca de busca de código aberto escrita em **Java**, amplamente utilizada como base para outros motores de busca. Diferentemente das demais soluções apresentadas nesta seção, o Lucene não opera como um serviço independente: ele é uma biblioteca que deve ser integrada diretamente ao código da aplicação, oferecendo maior controle sobre indexação, análise de texto e ranqueamento, porém exigindo maior esforço de desenvolvimento para sua utilização.

4.3.2 Apache Solr

O **Apache Solr** (Apache Software Foundation, 2025b) é um motor de busca de código aberto escrito em **Java**, construído sobre o Apache Lucene. Assim como o Elasticsearch, opera como um *webservice*: mantém um servidor standalone que expõe uma API REST para indexação e consulta de documentos. Oferece recursos satisfatórios de busca textual, análise linguística e suporte a múltiplos formatos de documento, podendo ser amplamente adotado em sistemas corporativos e bibliotecas digitais.

4.3.3 Vespa

O **Vespa** (Vespa.ai, 2025b) é uma plataforma de busca de código aberto desenvolvida originalmente pelo **Yahoo!** e posteriormente tornada pública (Vespa.ai, 2025a), escrita predominantemente em Java e C++. Também opera como um *webservice* distribuído, com diferencial no suporte nativo a busca vetorial (ANN) e ranqueamento com modelos de *machine learning* (TensorFlow, PyTorch, ONNX), sendo utilizado por empresas como Spotify e Yahoo em aplicações de busca e recomendação em larga escala (Vespa.ai, 2025a). Para o contexto deste trabalho, sua complexidade de configuração e infraestrutura seria desproporcional à coleção de atas institucionais da UFRRJ.

4.3.4 Elasticsearch

O **Elasticsearch** (IBM, 2023b) é um motor de busca e análise de código aberto, baseado na biblioteca Apache Lucene. Oferece recursos de busca escaláveis, podendo armazenar dados estruturados em formatos especializados para buscas otimizadas. Possui também um design de API RESTful, o que proporciona flexibilidade para a análise de dados e facilidade para a integração com aplicações. Trata-se de uma solução de busca textual rápida e amplamente disseminada, com escalabilidade horizontal e compatibilidade com diversas linguagens de programação, incluindo Java, Python, .NET, PHP, entre outras, e se destaca nesse cenário por combinar essas características com suporte a análise morfológica e o algoritmo BM25 para ranqueamento por relevância textual, permitindo a criação de analisadores personalizados a fim de aumentar a precisão da busca.

A escolha pelo Elasticsearch foi reforçada pela existência de trabalhos anteriores que buscam solucionar o mesmo problema no contexto institucional da UFRRJ. Em particular, o trabalho *CapiBusca: um buscador de atas para o Departamento de Ciência da Computação* (RIGONI, 2023), desenvolvido por Vinicius Nunes Rigoni em 2023, que já havia adotado o Elasticsearch como motor de busca para indexação e consulta das atas do DCC, validando a aplicabilidade também para o nosso cenário. Dessa forma, além de suas características técnicas, a existência de trabalhos

anteriores no mesmo contexto permitiu concentrar os esforços deste trabalho na camada conversacional e na geração estruturada de consultas.

4.4 Indexador de Documentos

O indexador foi desenvolvido em **Node.js** (OpenJS Foundation, 2025b), utilizando o framework **Express** (OpenJS Foundation, 2025a) para expor uma API REST com três rotas principais:

- **/indexar**: percorre a pasta de documentos configurada, extrai o texto de cada arquivo PDF por meio da biblioteca **pdf-parse** (npm, 2025b) e envia o conteúdo extraído ao Elasticsearch;
- **/buscar**: recebe os critérios de busca informados pelo usuário (discente, docente, disciplina, processo, progressão, assunto, departamento e intervalo de datas) e monta dinamicamente uma consulta no formato *Query DSL* do Elasticsearch, combinando cláusulas **match** e **match_phrase** com pesos (*boost*) distintos para cada campo, de modo a priorizar os critérios mais específicos;
- **/arquivos**: disponibiliza o acesso direto aos arquivos PDF originais por meio de uma rota estática.

A comunicação entre o indexador e o Elasticsearch é realizada por meio do cliente oficial **@elastic/elasticsearch** (Elastic, 2025), configurado com autenticação por usuário e senha e com validação do certificado TLS do servidor. As configurações sensíveis, como a URL do Elasticsearch, as credenciais de acesso, o caminho da pasta de documentos e o nome do índice utilizado, são definidas em variáveis de ambiente, carregadas por meio da biblioteca **dotenv** (npm, 2025a) a partir de um arquivo **.env**, evitando que informações sensíveis fiquem expostas diretamente no código-fonte.

O processo de indexação foi projetado para ser resiliente a falhas pontuais: cada arquivo da pasta de documentos é processado individualmente, dentro de um bloco de tratamento de erros próprio, de modo que a falha na leitura ou na extração de

texto de um arquivo específico (por exemplo, um PDF corrompido ou protegido) é registrada em log, mas não interrompe o processamento dos demais arquivos do lote. Arquivos dos quais não é possível extrair conteúdo textual são simplesmente ignorados, sem gerar um documento vazio no índice. Ao final do processamento de todos os arquivos, é solicitada uma operação de *refresh* do índice no Elasticsearch, tornando os documentos recém-indexados imediatamente disponíveis para consulta, sem a necessidade de aguardar o intervalo padrão de atualização do índice.

O código-fonte completo do indexador, assim como dos demais componentes do sistema, está disponível no repositório do projeto no GitHub (SILVA; VASCONCELOS, 2026).

4.5 Mapeamento dos Critérios de Busca para a Consulta

Esta seção detalha a implementação do roteiro guiado de diálogo apresentado no Capítulo 3, descrevendo como cada critério de busca coletado pelo chatbot é convertido em parâmetros da requisição ao indexador e, em seguida, em cláusulas da consulta ao Elasticsearch.

4.5.1 Critérios de Busca e Variáveis de Sessão

O fluxo principal do chatbot, definido no arquivo `main.flow.json` do Botpress, conduz o usuário por sete critérios de busca, cada um armazenado em uma variável de sessão e, posteriormente, repassado como parâmetro do endpoint `/buscar` do indexador:

- **Departamento** : nome ou sigla do departamento responsável pela ata, mapeado para os parâmetros `nomeDpto`;
- **Disciplina** : nome da disciplina mencionada na ata, mapeado para o parâmetro `nomeDisciplinas`;
- **Docentes** : nome de professores envolvidos no assunto da ata, mapeado para o parâmetro `nomeProfessores`;

- **Período** : intervalo de datas da ata, convertido para os parâmetros `de` e `ate`;
- **Discentes** : nome de alunos mencionados na ata, mapeado para o parâmetro `nomeDiscentes`;
- **Processo/Edital** : número de processo ou edital, mapeado para o parâmetro `numeroProcessoEdital`;
- **Progressão** : termo relacionado a progressões funcionais/acadêmicas, mapeado para o parâmetro `progressoes`.

Além desses sete critérios, o fluxo reserva um campo de **Outros Assuntos**, mapeado para o parâmetro `outrosAssuntos`, utilizado para descrições livres do tema da ata relacionados ao escopo acadêmico.

4.5.2 Montagem da Consulta ao Elasticsearch

O endpoint `/buscar` do indexador recebe os parâmetros correspondentes aos critérios informados pelo usuário durante a interação com o chatbot e monta dinamicamente uma consulta no formato *Query DSL* do Elasticsearch. Os seis parâmetros definidos na constante `CAMPOS_TEXTO` (`nomeDiscentes`, `nomeProfessores`, `nomeDisciplina`, `numeroProcessoEdital`, `progressoes` e `outrosAssuntos`) dão origem, cada um, a uma cláusula `multi_match` pesquisada simultaneamente sobre quatro campos do documento indexado (`conteudo`, `texto_da_ata`, `texto` e `ementa`), com peso (*boost*) próprio, definido de acordo com a especificidade do critério: `nomeDiscentes` e `nomeProfessores` recebem *boost* 2; `nomeDisciplina` e `numeroProcessoEdital` recebem *boost* 1,5; e `progressoes` e `outrosAssuntos` recebem *boost* 1, com tolerância a erros de digitação (`fuzziness: AUTO`). O departamento (`nomeDepto`) é tratado em uma cláusula `multi_match` própria, com *boost* 1,5 e o operador `or` entre os termos da consulta, permitindo que o nome ou a sigla do departamento sejam reconhecidos de forma mais flexível. Todas essas cláusulas compõem o agrupamento `should` da consulta booleana, com `minimum_should_match` igual a 1, garantindo que ao menos um dos critérios informados seja correspondido.

A integração entre o indexador e o Elasticsearch é viabilizada pela própria API RESTful do motor de busca, que conta com clientes oficiais para diversas linguagens, como Java, Python, .NET, PHP e JavaScript/Node.js. Neste projeto, essa integração é feita por meio do cliente oficial **@elastic/elasticsearch** (Elastic, 2025), já apresentado na Seção 4.4, responsável por traduzir a estrutura de consulta montada pelo endpoint `/buscar` para requisições no formato *Query DSL* aceito pelo servidor. A consulta resultante, no formato *Query DSL* do Elasticsearch, é ilustrada a seguir.

```
QUERY CONSOLIDADA ENVIADA AO ELASTICSEARCH:
{
  "query": {
    "bool": {
      "should": [
        {
          "multi_match": {
            "query": "Rodrigo Amado",
            "fields": [
              "conteudo",
              "texto_da_ata",
              "texto",
              "ementa"
            ],
            "boost": 2
          }
        },
        {
          "multi_match": {
            "query": "Departamento de Ciência da Computação",
            "fields": [
              "conteudo",
              "texto_da_ata",
              "texto",
              "ementa"
            ],
            "boost": 1.5,
            "operator": "or"
          }
        }
      ],
      "minimum_should_match": 1
    }
  },
  "query_string": {
    "query": "dezembro* AND 2025*",
    "fields": [
      "conteudo",
      "texto_da_ata",
      "texto",
      "ementa"
    ],
    "boost": 1.2
  }
}
```

Figura 4.1: Query gerada com os seguintes parâmetros utilizados: Departamento, Discente e Período

4.5.3 Seleção e Exibição dos Resultados

A ação `buscarAtasProporcionais` é responsável por montar a requisição HTTP ao endpoint `/buscar` a partir das variáveis de sessão coletadas ao longo do roteiro de diálogo e por processar a resposta do indexador. Cada critério informado pelo usuário (departamento, disciplina, docentes, discentes, processo/edital, período, progressão e assunto geral) é incluído como parâmetro da *query string* da requisição apenas quando preenchido; caso nenhum critério tenha sido informado, a busca não é realizada e a ação é encerrada antecipadamente, sinalizando que não houve resultado.

A requisição é feita por meio da biblioteca **axios** (axios, 2025), com um tempo limite de cinco segundos. Os documentos retornados pelo indexador são ordenados de forma decrescente pelo *score* de relevância atribuído pelo Elasticsearch, e os três primeiros colocados são selecionados para exibição. Para cada um desses documentos, a ação monta uma mensagem formatada contendo o título da ata, o *score* (arredondado para duas casas decimais) e o *link* de acesso ao arquivo PDF correspondente, servido pela rota `/arquivos` do indexador. Como o Botpress interpreta o caractere *underscore* (`_`) como marcação de itálico no formato Markdown, os títulos dos documentos — que seguem o padrão de nomenclatura das atas, geralmente com *underscores* no lugar de espaços — têm esse caractere precedido por uma barra invertida antes da formatação da mensagem, evitando que o nome do arquivo seja exibido de forma incorreta ao usuário.

Caso a busca não retorne nenhum documento, é exibida uma mensagem informando que nenhuma ata foi encontrada para os critérios informados; caso ocorra um erro durante a requisição ao indexador (por exemplo, indisponibilidade do serviço), é exibida uma mensagem genérica de erro ao usuário. O retorno das atas pelo buscador é exibido ao usuário conforme figura abaixo.

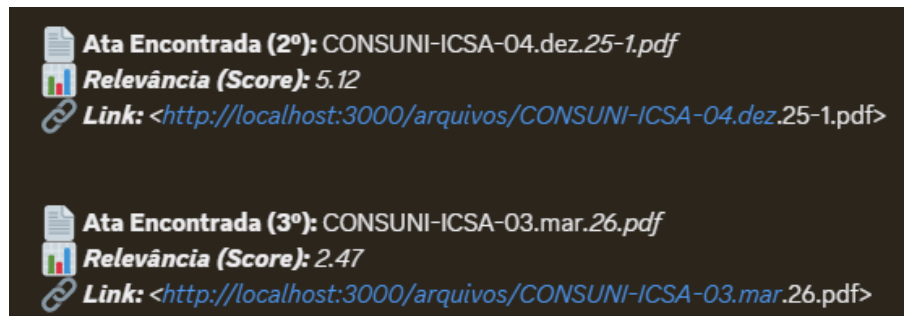


Figura 4.2: Retorno do bot com as atas encontradas para a busca realizada.

4.6 Integração com o Discord

Como canal alternativo de acesso ao chatbot, foi desenvolvido um bot para o **Discord** utilizando a biblioteca **discord.js** (discord.js, 2025). Essa integração visa facilitar a utilização do bot pela comunidade acadêmica, visto que o DCC utiliza o discord como um dos canais oficiais de comunicação.

O bot escuta as mensagens enviadas em um canal específico e repassa o conteúdo para o Botpress por meio de uma requisição HTTP feita com a biblioteca **axios** (axios, 2025), retornando ao usuário as respostas e os menus de opções gerados pelo fluxo de conversa.

Código 4.1: Escuta de mensagens no Discord e repasse ao Botpress via axios

```
client.on('messageCreate', async (message) => {  
  if (message.author.bot) return;  
  if (message.channelId !== CANAL_PERMITIDO_ID) return;  
  
  let perguntaUsuario = message.content.trim();  
  const usuarioId = message.author.id;  
  
  try {  
    const urlBotpress = `http://localhost:${BOTPRESS_PORT}/api/  
      v1/bots/${BOT_ID}/converse/${usuarioId}`;  
    const response = await axios.post(urlBotpress, { type: '  
      text', text: perguntaUsuario });  
  
    // ... processamento da resposta do Botpress ...  
  }  
});
```

```
    } catch (error) {  
        console.error('Erro de comunicacao com o Botpress:', error.  
            message);  
        await message.reply('Ocorreu um erro ao consultar o  
            assistente.');
```

Quando a resposta do Botpress contém um menu de opções (campos `choices`, `options` ou `quick_replies` da resposta da API), o bot extrai o título de cada opção e monta uma mensagem numerada no Discord. Essa lista de títulos é, então, armazenada em uma estrutura de memória temporária, associada ao identificador do usuário no Discord. Dessa forma, quando o usuário responde a um menu digitando apenas o número correspondente a uma das opções exibidas, o bot verifica se há uma lista de opções salva para aquele usuário e, em caso positivo, converte o número informado de volta para o texto original da opção escolhida, antes de repassar a mensagem ao Botpress. Esse mecanismo permite que o usuário interaja com os menus de opções do fluxo de diálogo digitando apenas números, sem que o Discord, diferentemente do Botpress, ofereça suporte nativo a botões ou outros elementos interativos de seleção. Essa ação visa facilitar a interação do usuário com o bot, sem que ele necessite escrever o nome da opção que deseja selecionar a cada interação. Abaixo temos um exemplo do menu exibido ao usuário.

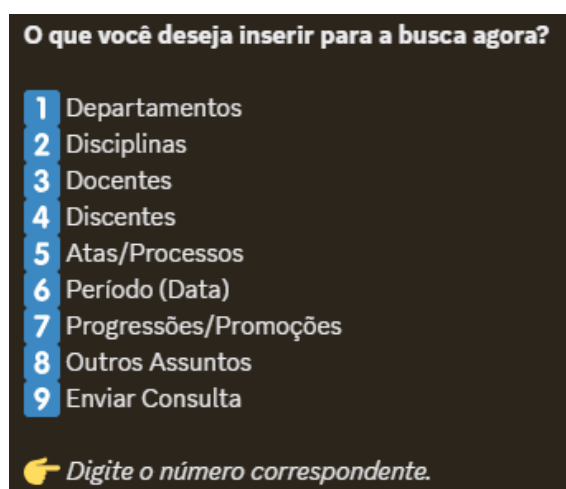


Figura 4.3: Menu exibido ao usuário para facilitar a interação com o bot.

4.7 Ambiente de Desenvolvimento

O indexador e o bot do Discord exigem o **Node.js** na versão 20 ou superior. Essa exigência decorre do uso do prefixo `node:` em módulos internos e de funcionalidades da *Fetch API* (como a classe `File`), utilizadas internamente pelas dependências do projeto e disponíveis apenas a partir dessa versão; em versões anteriores do Node.js, a execução do indexador resulta em erros de módulo não encontrado. As dependências do projeto (`express`, `pdf-parse`, `@elastic/elasticsearch`, `dotenv`, `discord.js` e `axios`) são gerenciadas por meio do **npm** e descritas no arquivo `package.json`.

4.8 Utilização do Chatbot

Conforme discutido nas Seções 3.1.3 e 4.5, o usuário possui um escopo fechado de critérios de busca, sendo possível pesquisar nas atas pelos seguintes temas: departamento, disciplina, docentes, discentes, período, número de ata, processo ou edital e progressões funcionais, além de ser possível informar outros assuntos relevantes no contexto acadêmico que não tenham sido mapeados nos critérios anteriores. Esta seção ilustra, por meio de capturas de tela reais da interação no Discord, como esse escopo se traduz na experiência do usuário.

4.8.1 Abertura da Conversa

Em sua primeira interação, o bot pergunta o nome do usuário a fim de tornar as interações seguintes um pouco mais pessoais (Figura 4.4).



Figura 4.4: Primeira interação com o bot, na qual é informado o nome do usuário.

Na interação seguinte, o bot exibe um menu com três opções, a partir do qual o usuário pode prosseguir diretamente para o início da busca, consultar instruções detalhadas de uso ou obter informações sobre o trabalho (Figura 4.5). Caso opte pelas instruções de uso, uma nova mensagem é enviada explicando como cada critério de busca deve ser informado, deixando claro que não é necessário preencher todos os campos, cabendo ao usuário informar apenas aquilo que for relevante para sua pesquisa; em seguida, o mesmo menu de três opções é novamente exibido, permitindo que o usuário prossiga para o início da busca.

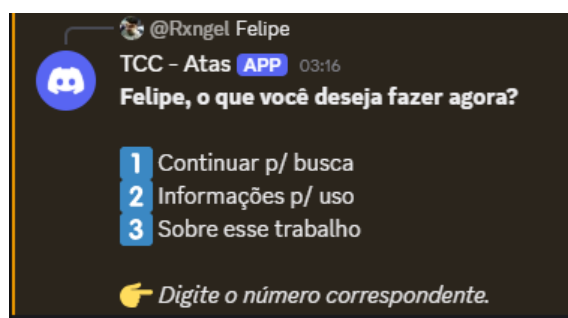


Figura 4.5: Menu de opções inicial, a partir do qual é possível iniciar a busca ou receber instruções de uso.

4.8.2 Navegação pelos Critérios de Busca

Uma vez iniciada a busca, o bot apresenta um menu com os oito critérios de busca disponíveis, além da opção de enviar a consulta (Figura 4.6). A cada critério informado pelo usuário, esse mesmo menu é exibido novamente, permitindo que outro

critério seja preenchido ou que a consulta seja enviada; vale destacar que, como a consulta ao Elasticsearch é montada apenas ao final da interação, consolidando todas as informações fornecidas, o usuário deve informar todos os critérios desejados antes de optar por enviar a consulta, e não a cada critério individualmente.

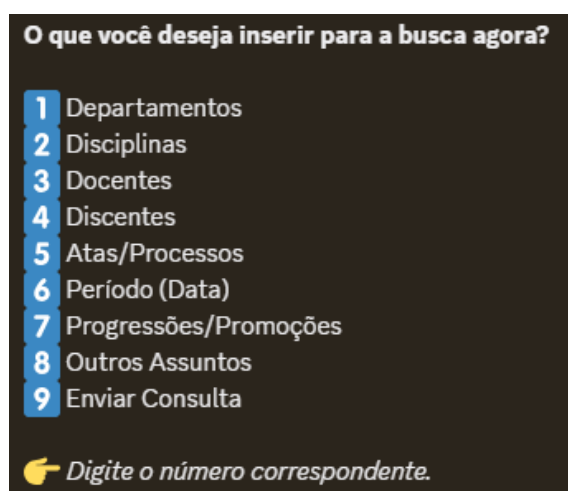


Figura 4.6: Menu de opções para navegação entre os critérios de busca.

4.8.3 Consolidação das Informações e Envio da Consulta

Ao selecionar a opção de enviar a consulta, o bot apresenta uma mensagem de consolidação com todas as respostas informadas até aquele momento, permitindo que o usuário confira exatamente o que será utilizado na busca antes de submetê-la (Figura 4.7). No exemplo ilustrado, foram informados os critérios de departamento, discente e período.

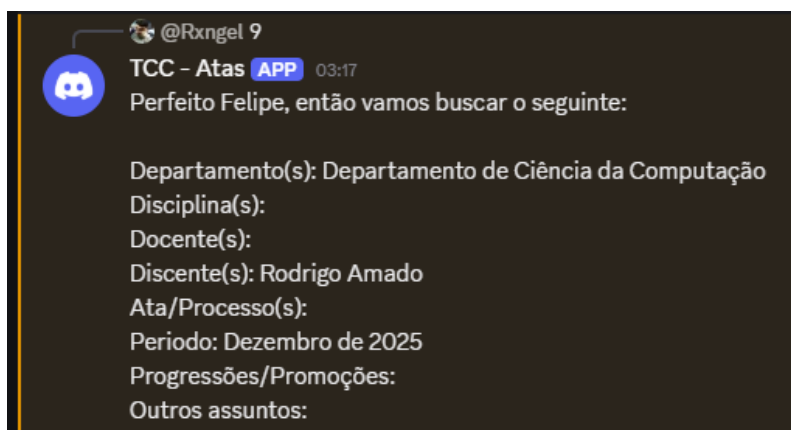


Figura 4.7: Consolidação das respostas informadas pelo usuário antes do envio da consulta.

A partir dessas mesmas informações, o indexador monta a consulta no formato *Query DSL* enviada ao Elasticsearch, conforme descrito na Seção 4.5. A Figura 4.8 ilustra a consulta efetivamente gerada para o exemplo da Figura 4.7: o nome do discente origina uma cláusula `multi_match` com *boost* 2, o nome do departamento origina uma cláusula `multi_match` com *boost* 1,5 e operador `or`, e o período origina uma cláusula `query_string` com termos providos de curinga e *boost* 1,2, todas agrupadas em uma cláusula `should` com `minimum_should_match` igual a 1.

```
QUERY CONSOLIDADA ENVIADA AO ELASTICSEARCH:
{
  "query": {
    "bool": {
      "should": [
        {
          "multi_match": {
            "query": "Rodrigo Amado",
            "fields": [
              "conteudo",
              "texto_da_ata",
              "texto",
              "ementa"
            ],
            "boost": 2
          }
        },
        {
          "multi_match": {
            "query": "Departamento de Ciência da Computação",
            "fields": [
              "conteudo",
              "texto_da_ata",
              "texto",
              "ementa"
            ],
            "boost": 1.5,
            "operator": "or"
          }
        },
        {
          "query_string": {
            "query": "dezembro* AND 2025*",
            "fields": [
              "conteudo",
              "texto_da_ata",
              "texto",
              "ementa"
            ],
            "boost": 1.2
          }
        }
      ],
      "minimum_should_match": 1
    }
  }
}
```

Figura 4.8: Query consolidada enviada ao Elasticsearch para o exemplo das Figuras 4.7 e 4.9.

4.8.4 Exibição dos Resultados

Após o envio da consulta, até três atas são exibidas ao usuário, ordenadas de forma decrescente de acordo com o *score* atribuído pelo Elasticsearch, conforme descrito na Seção 4.5.3 (Figura 4.9). Para cada ata, são exibidos o título do arquivo, o *score* de relevância e o *link* de acesso ao PDF correspondente, servido pela rota

/arquivos do indexador.

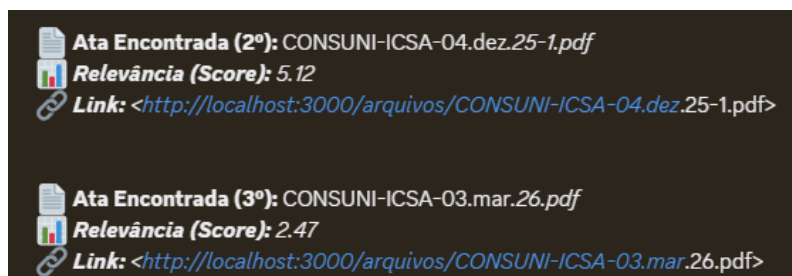


Figura 4.9: Retorno das atas encontradas, classificadas por *score*.

Capítulo 5

Conclusão

Este capítulo encerra o trabalho com um breve resumo do que foi apresentado, considerações sobre o que foi alcançado em relação ao objetivo proposto na Introdução (Capítulo 1), as limitações identificadas e sugestões para trabalhos futuros.

5.1 Considerações finais

Este trabalho partiu da necessidade de facilitar a busca por atas de reuniões da UFRRJ, propondo um chatbot de regras capaz de mapear as perguntas de discentes, docentes e do público acadêmico em geral para as atas mais adequadas ao conteúdo de suas buscas. Para isso, o Capítulo 2 situou o problema dentro da literatura sobre chatbots e sobre Busca e Recuperação da Informação (BRI), apresentando a taxonomia dos principais tipos de chatbot disponíveis, o comparativo entre essas opções e os requisitos do projeto, e os modelos clássicos de recuperação, com destaque para o modelo probabilístico e o algoritmo BM25, utilizado pelo Elasticsearch para o ranqueamento padrão de resultados.

A partir dessa fundamentação, o Capítulo 3 caracterizou o problema central deste trabalho: a desconexão entre o vocabulário livre empregado pelo usuário em uma conversa e a forma como o conteúdo das atas está organizado no índice do Elasticsearch. A solução proposta foi um roteiro guiado de diálogo, que conduz

o usuário por um conjunto fechado de critérios de busca, traduz suas respostas em parâmetros estruturados e delimita, de forma explícita, o que pode ou não ser buscado, fechando essa lacuna sem exigir que o usuário conheça a organização do índice ou a sintaxe de consulta do motor de busca.

O Capítulo 4 descreveu a implementação dessa proposta, composta pelo Botpress, responsável pelo fluxo de diálogo; pelo indexador, desenvolvido em Node.js com Express, responsável por extrair o conteúdo das atas em PDF e montar a consulta ao Elasticsearch; pelo Elasticsearch propriamente dito, responsável pelo armazenamento e pelo ranqueamento via BM25; e por um bot do Discord, oferecido como canal alternativo de interação. Foi detalhado o mapeamento de cada um dos sete critérios de busca do roteiro (departamento, disciplina, docentes, discentes, período, processo/edital e progressão), além do campo de outros assuntos, para cláusulas da consulta em *Query DSL*, com pesos distintos conforme a especificidade de cada critério, e a aplicação do score para reordenar os três melhores resultados antes de sua exibição ao usuário.

Em relação ao objetivo definido na Introdução, definido por disponibilizar uma ferramenta conversacional capaz de interpretar perguntas em linguagem natural e mapeá-las para as atas institucionais mais pertinentes, o trabalho alcançou esse resultado por meio da combinação de um chatbot baseado em regras com um roteiro guiado de diálogo, com as definições dos parâmetros que seriam oferecidos para o usuário realizar a personalização da busca. O comparativo do Capítulo 2 (Tabela 2.1) evidenciou que a escolha por um chatbot baseado em regras é coerente com o escopo fechado do problema, com a ausência de orçamento para licenciamento de ferramentas e com a necessidade de integração via API com o Elasticsearch. O roteiro guiado de diálogo, por sua vez, mostrou-se uma solução viável para o problema de geração de consultas a partir de uma interação conversacional, ao definir de forma explícita quais critérios podem ser buscados e como cada resposta do usuário se traduz em um componente da consulta, sem exigir do usuário qualquer conhecimento sobre a organização do índice ou sobre o BM25.

5.2 Limitações e trabalhos futuros

Por se tratar de um chatbot baseado em regras, o sistema opera sobre um vocabulário e um conjunto de critérios de busca fechados e definidos previamente no roteiro de diálogo. Conforme discutido nos Capítulos 2 e 3, essa característica é adequada ao escopo do problema, mas limita a interpretação de perguntas que não se encaixem nos sete critérios previstos (departamento, disciplina, docentes, discentes, período, processo/edital e progressão) ou no campo de assunto geral, o qual concentra qualquer informação fornecida pelo usuário que não corresponda a esses critérios ou a um período de datas reconhecível. Como trabalho futuro, sugere-se o mapeamento de novos critérios de busca além dos sete atualmente previstos, a partir do levantamento de outros aspectos recorrentes no conteúdo das atas institucionais que ainda não tenham sido contemplados pelo roteiro — por exemplo, tipos específicos de deliberação ou categorias de assunto frequentemente tratadas pelo colegiado — de modo a reduzir, ao longo do tempo, a dependência do campo de assunto geral para temas que poderiam ser tratados como critérios próprios, com peso e tipo de correspondência específicos na consulta ao Elasticsearch.

Uma extensão mais ampla seria a evolução do chatbot atual, baseado exclusivamente em regras, para um modelo híbrido, que combine o roteiro guiado já implementado com uma camada de IA generativa para a interpretação de perguntas em linguagem mais livre, sobretudo naquelas que hoje são apenas direcionadas ao campo de assunto geral. Uma implementação desse tipo, no entanto, precisaria de um cuidado adicional para mitigar o risco de alucinação já discutido na Seção 3.1.3: a geração de respostas pelo componente de IA deveria ser estritamente fundamentada no conteúdo efetivamente recuperado do Elasticsearch (uma arquitetura de *Retrieval-Augmented Generation*, ou RAG), com instruções de contexto bem definidas que restrinjam o modelo a responder apenas com base nas atas retornadas pela busca, evitando que o chatbot produza informações não embasadas nos documentos institucionais, como discutido no exemplo do chatbot da concessionária Chevrolet de Watsonville (AI Incident Database, 2023; VentureBeat, 2023).

O sistema também está, atualmente, restrito ao Botpress e ao bot do Discord

como canais de interação. Como trabalho futuro, é possível integrar o mesmo roteiro de diálogo a outras plataformas de uso comum pela comunidade acadêmica, como um *widget* de chat no próprio site da universidade ou do departamento, ou um bot para o Telegram, ampliando o alcance do sistema sem a necessidade de duplicar a lógica de busca já implementada no indexador, dado que o Botpress permite a integração com múltiplos canais de comunicação a partir de um único fluxo de diálogo.

Por fim, no que diz respeito ao armazenamento dos documentos, as atas em PDF utilizadas neste trabalho são mantidas localmente, em uma pasta no mesmo ambiente onde o indexador é executado, sendo apenas seu conteúdo textual extraído e enviado ao Elasticsearch. Essa abordagem é suficiente para o escopo deste trabalho, mas limita a escalabilidade e a disponibilidade do sistema a um único ambiente local. Como trabalho futuro, sugere-se a migração do armazenamento das atas para um serviço de armazenamento em nuvem ou para um banco de dados próprio para arquivos, permitindo que o indexador e a rota de acesso aos arquivos PDF (`/arquivos`) passem a consultar as atas a partir desse serviço externo, em vez de um diretório local, o que facilitaria tanto a hospedagem do sistema em um ambiente de produção quanto a atualização do acervo de atas por outros membros do departamento.

Sugere-se ainda, como trabalho futuro, a realização de uma avaliação de usabilidade com usuários reais do Departamento de Ciência da Computação da UFRRJ, envolvendo discentes, docentes e o público acadêmico em geral, de modo a verificar, em uso real, a efetividade do roteiro guiado na recuperação das atas desejadas e a identificar critérios de busca ou formulações de pergunta não previstos no escopo atual.

Referências

ADAMOPOULOU, E.; MOUSSIADES, L. Chatbots: History, technology, and applications. *Machine Learning with Applications*, v. 2, p. 100006, 2020.

AI Incident Database. *Incident 622: Chevrolet Dealer Chatbot Agrees to Sell Tahoe for \$1*. 2023. Disponível em: <<https://incidentdatabase.ai/cite/622/>>. Acesso em: 20 jun. 2026.

American Bar Association. *BC Tribunal Confirms Companies Remain Liable for Information Provided by AI Chatbot*. 2024. Disponível em: <https://www.americanbar.org/groups/business_law/resources/business-law-today/2024-february/bc-tribunal-confirms-companies-remain-liable-information-provided-ai-chatbot/>. Acesso em: 18 jun. 2026.

Apache Software Foundation. *Apache Lucene Core*. 2025. Disponível em: <<https://lucene.apache.org/core/>>. Acesso em: 17 jun. 2026.

Apache Software Foundation. *Apache Solr*. 2025. Disponível em: <<https://solr.apache.org/>>. Acesso em: 17 jun. 2026.

ARAUJO, E. F. dos S. Solução chatbot no ambiente acadêmico da ufrj. 2020.

axios. *Axios - Promise based HTTP client for the browser and node.js*. 2025. Disponível em: <<https://axios-http.com/>>. Acesso em: 09 jun. 2026.

BAEZA-YATES, R.; RIBEIRO-NETO, B. *Modern Information Retrieval: The Concepts and Technology Behind Search*. 2nd. ed. [S.l.]: Addison-Wesley, 2010.

BARBOSA, L. S. Um chatbot especializado para o contexto da universidade federal de ouro preto. 2023.

BOMMASANI, R.; AL. et. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

Botpress. *About Botpress*. 2025. Disponível em: <<https://botpress.com/about>>. Acesso em: 07 jul. 2025.

Botpress. *Botpress Documentation – v12*. 2025. Disponível em: <<https://botpress.com/docs/v12/>>. Acesso em: 07 jul. 2025.

CBC News. *How can I mislead you? Air Canada found liable for chatbot's bad advice on bereavement rates*. 2024. Disponível em: <<https://www.cbc.ca/news/canada/british-columbia/air-canada-chatbot-lawsuit-1.7116416>>. Acesso em: 18 jun. 2026.

COLBY, K. M. et al. Turing-like indistinguishability tests for the validation of a computer simulation of paranoid processes. *Artificial Intelligence*, v. 3, p. 199–221, 1972.

DALE, R. The return of the chatbots. *Natural Language Engineering*, v. 22, n. 5, p. 811–817, 2016.

discord.js. *discord.js Documentation*. 2025. Disponível em: <<https://discord.js.org/>>. Acesso em: 09 jun. 2026.

Elastic. *Elasticsearch JavaScript Client*. 2025. Disponível em: <<https://www.elastic.co/guide/en/elasticsearch/client/javascript-api/current/introduction.html>>. Acesso em: 09 jun. 2026.

Flow XO. *Pricing*. 2026. Disponível em: <<https://flowxo.com/pricing/>>. Acesso em: 20 jun. 2026.

Google Cloud. *Dialogflow Quotas and Limits*. 2026. Disponível em: <<https://docs.cloud.google.com/dialogflow/quotas>>. Acesso em: 20 jun. 2026.

HOY, M. B. Alexa, siri, cortana, and more: An introduction to voice assistants. *Medical Reference Services Quarterly*, v. 37, n. 1, p. 81–88, 2018.

IBM. *O que é LLM (large language models)?* 2023. Disponível em: <<https://www.ibm.com/br-pt/think/topics/large-language-models>>. Atualizado em 02/11/2023.

IBM. *O que é o Elasticsearch?* 2023. Disponível em: <<https://www.ibm.com/br-pt/think/topics/elasticsearch>>. Atualizado em 29/09/2021.

IBM. *Seis tipos de chatbots e como escolher o ideal para sua empresa*. 2025. Disponível em: <<https://www.ibm.com/br-pt/think/topics/chatbot-types>>. Atualizado em 07/03/2025.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZ, H. *Introduction to Information Retrieval*. Cambridge: Cambridge University Press, 2008.

npm. *dotenv*. 2025. Disponível em: <<https://www.npmjs.com/package/dotenv>>. Acesso em: 09 jun. 2026.

npm. *pdf-parse*. 2025. Disponível em: <<https://www.npmjs.com/package/pdf-parse>>. Acesso em: 09 jun. 2026.

OpenJS Foundation. *Express - Node.js web application framework*. 2025. Disponível em: <<https://expressjs.com/>>. Acesso em: 09 jun. 2026.

OpenJS Foundation. *Node.js Documentation*. 2025. Disponível em: <<https://nodejs.org/en/docs>>. Acesso em: 09 jun. 2026.

Rasa. *Rasa Documentation*. 2026. Disponível em: <<https://rasa.com/docs/>>. Acesso em: 20 jun. 2026.

RIGONI, V. N. *CapiBusca: um buscador de atas para o Departamento de Ciência da Computação*. Tese (Monografia (Bacharelado em Ciência da Computação)) — Universidade Federal Rural do Rio de Janeiro, Nova Iguaçu, 2023.

ROBERTSON, S.; ZARAGOZA, H. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, v. 3, n. 4, p. 333–389, 2009.

Rocket.Chat. *Rocket.Chat Documentation*. 2026. Disponível em: <<https://docs.rocket.chat/>>. Acesso em: 20 jun. 2026.

SALTON, G.; WONG, A.; YANG, C.-S. A vector space model for automatic indexing. *Communications of the ACM*, v. 18, n. 11, p. 613–620, 1975.

SHUM, H.-Y.; HE, X.; LI, D. From eliza to xiaoice: Challenges and opportunities with social chatbots. *Frontiers of Information Technology & Electronic Engineering*, v. 19, n. 1, p. 10–26, 2018.

SILVA, F. R. d.; VASCONCELOS, G. P. V. d. *Chatbot para Atas Institucionais da UFRRJ*. 2026. Disponível em: <<https://github.com/EoqLp/TCC-Atas-UFRRJ>>. Acesso em: 20 jun. 2026.

SINGH, J.; JOESPH, M. H.; JABBAR, K. B. A. Rule-based chabot for student enquiries. *Journal of Physics: Conference Series*, v. 1228, n. 1, p. 012060, 2019.

TURING, A. M. Computing machinery and intelligence. *Mind*, LIX, n. 236, p. 433–460, October 1950.

VentureBeat. *A Chevy for \$1? Car dealer chatbots show perils of AI for customer service*. 2023. Disponível em: <<https://venturebeat.com/ai/a-chevy-for-1-car-dealer-chatbots-show-perils-of-ai-for-customer-service>>. Acesso em: 20 jun. 2026.

Vespa.ai. *Vespa – The AI Search Platform*. 2025. Disponível em: <<https://vespa.ai/>>. Acesso em: 17 jun. 2026.

Vespa.ai. *Vespa Overview*. 2025. Disponível em: <<https://docs.vespa.ai/en/learn/overview.html>>. Acesso em: 17 jun. 2026.

WALLACE, R. S. The anatomy of a.l.i.c.e. In: EPSTEIN, R.; ROBERTS, G.; BEBER, G. (Ed.). *Parsing the Turing Test: Philosophical and Methodological Issues in the Quest for the Thinking Computer*. [S.l.]: Springer, 2009. p. 181–210.

WEIZENBAUM, J. Eliza—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, v. 9, n. 1, p. 36–45, 1966.